

Projeto Multidisciplinar - Trilha Back-End

Rede "Raízes do Nordeste"

Aluno: [Wesley Guimarães Marinho / RU 4736850]

Polo de Apoio: [Araraquara]

Professor: Prof. Me. Luciane Yanase Kanashiro

Semestre/Ano: 2026

Sumário

1. Introdução
2. Análise e Requisitos
3. Modelagem e Arquitetura
4. API e Endpoints
5. LGPD, Privacidade e Segurança
6. Entrega Técnica
7. Plano de Testes
8. Conclusão
9. Referências

1. Introdução

Este trabalho apresenta o desenvolvimento da API (Back-End) para o estudo de caso da rede de lanchonetes "Raízes do Nordeste". Como o negócio está crescendo bastante e possui diferentes unidades, o foco do projeto foi criar um sistema capaz de receber pedidos de vários canais diferentes (App, Totem, Balcão, Web e Retirada) sem perder o controle e a rastreabilidade.

A ideia foi construir uma arquitetura funcional em Python, permitindo a gestão dos pedidos, o controle do cardápio e simulando a integração com um gateway de pagamentos externo (Mock). Tudo isso pensando em manter o código organizado e aplicar na prática os conceitos vistos durante o curso.

2. Análise e Requisitos

Requisitos Funcionais (RF)

- **RF01:** Cadastro e Autenticação de usuários, diferenciando os perfis em ADMIN, GERENTE e CLIENTE.
- **RF02:** Visualização de cardápio, permitindo a listagem e consulta detalhada de produtos.
- **RF03:** Realização de pedidos multicanal. A API deve exigir e registrar o canalPedido (APP, TOTEM, BALCAO, WEB, PICKUP).
- **RF04:** Atualização de status de pedido (CRIADO -> PAGO -> COZINHA -> ENTREGUE / CANCELADO).
- **RF05:** Mock de pagamento. O sistema deve validar a forma de pagamento externa, alterando o status caso seja "MOCK" ou simulando recusa.
- **RF06:** Controle de estoque com restrição de vendas, decrementando saldo ou bloqueando caso não haja produto.
- **RF07:** Fidelização (base: acumular pontos).

Requisitos Não Funcionais (RNF)

- **RNF01:** Segurança da informação. Uso de senhas criptografadas (Hash BCrypt) e JWT (JSON Web Tokens) para as chamadas de API.
- **RNF02:** Aderência à LGPD (coleta mínima de dados, finalidade justificada, hash de senhas e anonimização quando aplicável).
- **RNF03:** Arquitetura limpa/modular (divisão em Routers, Schemas, Models, Services).
- **RNF04:** API totalmente documentada via Swagger/OpenAPI.
- **RNF05:** Padrão uniforme de erro JSON para todos os endpoints.

3. Modelagem e Arquitetura

3.1. Diagrama de Casos de Uso (Conceitual)

- **Ator Cliente:** Realiza Pedido (via App/Web/Totem), Acompanha Pedido.
- **Ator Atendente:** Cadastra Produto, Lista Pedidos, Altera Status.
- **Ator Pagamento Externo (Gateway):** Processa e Confirma Transação.

3.2. Modelo de Dados (DER) e Diagrama de Classes

Principais Entidades:

- **Usuario** (id, nome, email, senha_hash, perfil)
- **Unidade** (id, nome, endereco)
- **Produto** (id, nome, descricao, preco, ativo)
- **Estoque** (id, produto_id [FK], unidade_id [FK], quantidade)
- **Pedido** (id, usuario_id [FK], canal_pedido, status, total)
- **ItemPedido** (id, pedido_id [FK], produto_id [FK], quantidade, preco_unitario)

3.3. Arquitetura em Camadas

O projeto utiliza a linguagem Python com o framework FastAPI, estruturando-se em:

- **API/Routers:** Camada de transporte (recebe requests, responde JSON).
- **Schemas:** Definição de contratos (Pydantic models) e validações.
- **Models/Infrastructure:** Definição das tabelas do banco (SQLAlchemy) e conexão.
- **Services/Domínio:** Onde apliquei as regras de negócio (cálculo de total, simulação do mock de pagamento, etc).

4. API e Endpoints

Abaixo estão os endpoints organizados. A documentação completa está acessível no `/docs` (Swagger).

Autenticação (`/auth`)

- **POST `/auth/login`:** Recebe {email, senha}, retorna {access_token, token_type}.
- **POST `/auth/registrar`:** Cria um novo usuário.

Produtos (`/produtos`)

- **GET `/produtos`:** Lista todos os produtos (pode filtrar).
- **POST `/produtos`:** Cria produto (somente ADMIN/GERENTE).

Pedidos (`/pedidos`)

- **POST `/pedidos`:** Cria um novo pedido. Requer canalPedido. Retorna 201 ou 422.
- **GET `/pedidos`:** Lista pedidos. Suporta `?canalPedido=APP`.
- **PUT `/pedidos/{id}/status`:** Atualiza o status do pedido e simula pagamento mock.

Padrão de Erro:

```
{
  "error": "VALIDATION_ERROR",
  "message": "Mensagem padronizada do erro.",
  "timestamp": "2026-06-23T17:00:00Z",
  "path": "/pedidos"
}
```

5. LGPD, Privacidade e Segurança no Back-End

Para atender aos requisitos de segurança e LGPD propostos pelo estudo de caso, procurei manter o armazenamento apenas do que é essencial para o fluxo funcionar.

- As senhas dos usuários nunca são salvas abertamente (usei a biblioteca nativa `bcrypt` para salvar apenas o hash no banco).
- Apliquei um esquema de autenticação com tokens JWT.
- Para fins de auditoria, a API gera logs de sistema sempre que uma ação sensível ocorre, como a criação ou a alteração de status de um pedido. Isso garante total rastreabilidade sobre as movimentações.
- Rotas de criação de produtos e controle mais sensível exigem que o usuário tenha o perfil de administrador ou gerente.
- Quando um cliente loga, ele consegue listar e visualizar apenas os seus próprios pedidos, evitando expor dados de outras pessoas e garantindo a privacidade da operação.

6. Entrega Técnica

- **Código-Fonte e Repositório:** O código encontra-se implementado e pode ser verificado no repositório.
- **Swagger Local:** Ao rodar a API, acessível via `http://localhost:8000/docs`.
- **Coleção Postman:** Disponibilizada como arquivo no diretório raiz do projeto.
- **README:** Contém as instruções de setup, inicialização e rodada de *migrations/seeds*.

7. Plano de Testes

Foram definidos cenários de teste na coleção Postman.

1. **T01 (Auth - Positivo):** Login válido (retorna 200 + token).
2. **T02 (Auth - Negativo):** Listar pedidos sem token (retorna 401).
3. **T03 (Produto - Positivo):** Listar produtos disponíveis (retorna 200 + array).
4. **T04 (Produto - Negativo):** Criar produto sem ser ADMIN (retorna 403).
5. **T05 (Pedido - Negativo):** Criar pedido sem informar `canalPedido` (retorna 422 Unprocessable Entity).
6. **T06 (Pedido - Positivo):** Criar pedido com itens válidos e canal APP (retorna 201 + dados).
7. **T07 (Pedido - Negativo):** Criar pedido de produto inexistente (retorna 404).
8. **T08 (Pagamento - Positivo):** Atualizar status para PAGO (Simulação de sucesso Gateway Mock) (retorna 200).
9. **T09 (Pagamento - Negativo):** Simulação de pagamento RECUSADO alterando forma de pagamento para Invalida (retorna 400).
10. **T10 (Busca - Positivo):** Buscar pedidos pelo filtro de canal `?canalPedido=APP` (retorna 200).

8. Conclusão

Desenvolver este projeto foi um ótimo desafio para colocar em prática o que foi ensinado na disciplina. Consegui montar uma API com FastAPI que não apenas atende aos requisitos do estudo de caso, como o uso do `canalPedido` e a simulação de pagamento via gateway (Mock), mas que também funciona de verdade integrado a um banco de dados relacional.

Optei por usar o SQLite para deixar o projeto mais fácil de ser executado e testado por quem for corrigir, sem perder a complexidade da estrutura de tabelas usando o SQLAlchemy. Acredito que o código ficou organizado, bem documentado através do Swagger automático e que essa solução se saíria muito bem em uma demonstração para vagas na área.

9. Referências

- Documentação do FastAPI. Disponível em: <https://fastapi.tiangolo.com/>
- Material de Eletiva IV - Projeto Multidisciplinar - UNINTER.